

MATHDNN_HW6

November 9, 2021

Problem 1

```
[1]: import torch.nn as nn
      from torch.utils.data import DataLoader
      import torch
      import torchvision
      import torchvision.transforms as transforms
```

```
[2]: # Instantiate model with BN and load trained parameters
      class smallNetTrain(nn.Module) :
          # CIFAR-10 data is 32*32 images with 3 RGB channels
          def __init__(self, input_dim=3*32*32) :
              super().__init__()

              self.conv1 = nn.Sequential(
                  nn.Conv2d(3, 16, kernel_size=3, padding=1),
                  nn.BatchNorm2d(16),
                  nn.ReLU()
              )

              self.conv2 = nn.Sequential(
                  nn.Conv2d(16, 16, kernel_size=3, padding=1),
                  nn.BatchNorm2d(16),
                  nn.ReLU()
              )

              self.fc1 = nn.Sequential(
                  nn.Linear(16*32*32, 32*32),
                  nn.BatchNorm1d(32*32),
                  nn.ReLU()
              )

              self.fc2 = nn.Sequential(
                  nn.Linear(32*32, 10),
                  nn.ReLU()
              )

          def forward(self, x) :
              x = self.conv1(x)
              x = self.conv2(x)
              x = x.float().view(-1, 16*32*32)
              x = self.fc1(x)
```

```

        x = self.fc2(x)

    return x

```

```

[4]: model = smallNetTrain()
model.load_state_dict(torch.load("./smallNetSaved",map_location=torch.
    ↪device('cpu')))

```

[4]: <All keys matched successfully>

```

[5]: # Instantiate model without BN
class smallNetTest(nn.Module) :
    # CIFAR-10 data is 32*32 images with 3 RGB channels
    def __init__(self, input_dim=3*32*32) :
        super().__init__()

        self.conv1 = nn.Sequential(
            nn.Conv2d(3, 16, kernel_size=3, padding=1),
            nn.ReLU()
        )
        self.conv2 = nn.Sequential(
            nn.Conv2d(16, 16, kernel_size=3, padding=1),
            nn.ReLU()
        )
        self.fc1 = nn.Sequential(
            nn.Linear(16*32*32, 32*32),
            nn.ReLU()
        )
        self.fc2 = nn.Sequential(
            nn.Linear(32*32, 10),
            nn.ReLU()
        )

    def forward(self, x) :
        x = self.conv1(x)
        x = self.conv2(x)
        x = x.float().view(-1, 16*32*32)
        x = self.fc1(x)
        x = self.fc2(x)

    return x

```

```

[6]: model_test = smallNetTest()

```

Exact Process

```

[7]: # Initialize weights of model without BN

conv1_bn_beta, conv1_bn_gamma = model.conv1[1].bias, model.conv1[1].weight

```

```

conv1_bn_mean, conv1_bn_var = model.conv1[1].running_mean, model.conv1[1].
    ↪running_var
conv2_bn_beta, conv2_bn_gamma = model.conv2[1].bias, model.conv2[1].weight
conv2_bn_mean, conv2_bn_var = model.conv2[1].running_mean, model.conv2[1].
    ↪running_var
fc1_bn_beta, fc1_bn_gamma = model.fc1[1].bias, model.fc1[1].weight
fc1_bn_mean, fc1_bn_var = model.fc1[1].running_mean, model.fc1[1].running_var
eps = 1e-05

# Initialize the following parameters

model_test.conv1[0].weight.data = (model.conv1[0].weight * conv1_bn_gamma.
    ↪reshape(-1, 1, 1, 1)) / torch.sqrt(conv1_bn_var.reshape(-1, 1, 1, 1) + eps)
model_test.conv1[0].bias.data = conv1_bn_beta + conv1_bn_gamma * (model.
    ↪conv1[0].bias - conv1_bn_mean) / torch.sqrt(conv1_bn_var + eps)
model_test.conv2[0].weight.data = (model.conv2[0].weight * conv2_bn_gamma.
    ↪reshape(-1, 1, 1, 1)) / torch.sqrt(conv2_bn_var.reshape(-1, 1, 1, 1) + eps)
model_test.conv2[0].bias.data = conv2_bn_beta + conv2_bn_gamma * (model.
    ↪conv2[0].bias - conv2_bn_mean) / torch.sqrt(conv2_bn_var + eps)
model_test.fc1[0].weight.data = (model.fc1[0].weight * fc1_bn_gamma.reshape(-1,
    ↪1)) / torch.sqrt(fc1_bn_var.reshape(-1, 1) + eps)
model_test.fc1[0].bias.data = fc1_bn_beta + fc1_bn_gamma * (model.fc1[0].bias -
    ↪fc1_bn_mean) / torch.sqrt(fc1_bn_var + eps)
model_test.fc2[0].weight.data = model.fc2[0].weight
model_test.fc2[0].bias.data = model.fc2[0].bias

# Verify difference between model and model_test

model.eval()
# model_test.eval() # not necessary since model_test has no BN or dropout

test_dataset = torchvision.datasets.CIFAR10(root='./cifar_10data/',
    train=False,
    transform=transforms.ToTensor(), download =
    ↪True)
test_loader = torch.utils.data.DataLoader(dataset=test_dataset, batch_size=100,
    ↪shuffle=False)

diff = []
with torch.no_grad():
    for images, _ in test_loader:
        diff.append(torch.norm(model(images) - model_test(images))**2)

```

```
print(max(diff)) # If less than 1e-08, you got the right answer.
```

Downloading <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz> to
./cifar_10data/cifar-10-python.tar.gz

0%| | 0/170498071 [00:00<?, ?it/s]

Extracting ./cifar_10data/cifar-10-python.tar.gz to ./cifar_10data/
tensor(6.9132e-09)

Previous Process Trial (For Reference)

```
[ ]: # Initialize weights of model without BN

conv1_bn_beta, conv1_bn_gamma = model.conv1[1].bias, model.conv1[1].weight
conv1_bn_mean, conv1_bn_var = model.conv1[1].running_mean, model.conv1[1].
    ↳running_var
conv2_bn_beta, conv2_bn_gamma = model.conv2[1].bias, model.conv2[1].weight
conv2_bn_mean, conv2_bn_var = model.conv2[1].running_mean, model.conv2[1].
    ↳running_var
fc1_bn_beta, fc1_bn_gamma = model.fc1[1].bias, model.fc1[1].weight
fc1_bn_mean, fc1_bn_var = model.fc1[1].running_mean, model.fc1[1].running_var
eps = 1e-05

def sixteen(x):
    return x.reshape([-1, 1, 1, 1])

# Initialize the following parameters
conv1_gamma_re = sixteen(conv1_bn_gamma)
conv1_bn_var+=eps
conv1_var_re = sixteen(conv1_bn_var)
model_test.conv1[0].weight.data = conv1_gamma_re*(model.conv1[0].weight.data/
    ↳torch.sqrt(conv1_var_re))
model_test.conv1[0].bias.data = conv1_bn_beta + conv1_bn_gamma*(model.conv1[0].
    ↳bias.data-conv1_bn_mean)/torch.sqrt(conv1_bn_var)

def f(x):
    return x.reshape([-1, 1, 1, 1])

conv2_gamma_re = f(conv2_bn_gamma)
conv2_bn_var+=eps
conv2_var_re = f(conv2_bn_var)
model_test.conv2[0].weight.data = conv2_gamma_re*(model.conv2[0].weight.data/
    ↳torch.sqrt(conv2_var_re))
model_test.conv2[0].bias.data = conv2_bn_beta + conv2_bn_gamma*(model.conv2[0].
    ↳bias.data-conv2_bn_mean)/torch.sqrt((conv2_bn_var))

def g(x):
```

```

    return x.reshape([-1, 1])

fc1_gamma_re = g(fc1_bn_gamma)
fc1_bn_var+=eps
fc1_var_re = g(fc1_bn_var)
model_test.fc1[0].weight.data = fc1_gamma_re*(model.fc1[0].weight.data/torch.
    ↪sqrt(fc1_var_re))
model_test.fc1[0].bias.data = fc1_bn_beta + fc1_bn_gamma*(model.fc1[0].bias.
    ↪data-fc1_bn_mean)/torch.sqrt((fc1_bn_var))

model_test.fc2[0].weight.data = model.fc2[0].weight.data
model_test.fc2[0].bias.data = model.fc2[0].bias.data

# Verify difference between model and model_test

model.eval()
# model_test.eval() # not necessary since model_test has no BN or dropout

test_dataset = torchvision.datasets.CIFAR10(root='./cifar_10data/',
                                             train=False,
                                             transform=transforms.ToTensor(), download =
    ↪True)
test_loader = torch.utils.data.DataLoader(dataset=test_dataset, batch_size=100,
    ↪shuffle=False)

diff = []
with torch.no_grad():
    for images, _ in test_loader:
        diff.append(torch.norm(model(images) - model_test(images))**2)

print(max(diff)) # If less than 1e-08, you got the right answer.

'''
For debugging purposes, you may want to match the output of conv1 first before
moving on working on conv2. To do so, you can replace the forward-evaluation
functions of the two models with
def forward(self, x) :
    x = self.conv1(x)
    return x
'''

```

Downloading <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz> to
./cifar_10data/cifar-10-python.tar.gz

```
0%|          | 0/170498071 [00:00<?, ?it/s]
```

```
Extracting ./cifar_10data/cifar-10-python.tar.gz to ./cifar_10data/  
tensor(5.5245e-06)
```

```
[ ]: '\nFor debugging purposes, you may want to match the output of conv1 first  
before\nmoving on working on conv2. To do so, you can replace the forward-  
evaluation\nfunctions of the two models with \ndef forward(self, x) :\n    x =  
self.conv1(x)\n    return x\n'
```

[Assignment VI]

Problem 2: Default weight initialization. Consider the multi-layer perceptron

$$\begin{aligned}
 y_L &= A_L y_{L-1} + b_L \\
 y_{L-1} &= \sigma(A_{L-1} y_{L-2} + b_{L-1}) \\
 &\vdots \\
 y_2 &= \sigma(A_2 y_1 + b_2) \\
 y_1 &= \sigma(A_1 x + b_1),
 \end{aligned}$$

$$y = \sum A_{L,i,j} y_{L-1,j} + b_L$$

where $x \in \mathbb{R}^{n_0}$, $A_\ell \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$, $b_\ell \in \mathbb{R}^{n_\ell}$, and $n_L = 1$. For the sake of simplicity, let

$$\sigma(z) = z. \quad x \sim N(0, 1)$$

Assume $x_1, \dots, x_{n_0}$ are IID with zero-mean and unit variance. If this network is initialized with the default weight initialization of PyTorch, what will the mean and variance of y_L be?

PyTorch default initialization;

$$y_{\ell,i} = \sum_j A_{\ell,i,j} y_{\ell-1,j} + b_i, \quad A_{\ell,i,j} \sim \text{Uni}\left(-\sqrt{\frac{1}{n_{\ell-1}}}, \sqrt{\frac{1}{n_{\ell-1}}}\right), \quad b_i \sim \text{Uni}\left(-\frac{1}{\sqrt{n_\ell}}, \frac{1}{\sqrt{n_\ell}}\right)$$

$$\mathbb{E}(y_{\ell,i}) = \sum_j \mathbb{E}(A_{\ell,i,j} y_{\ell-1,j}) + \mathbb{E}(b_i)$$

$$= \sum_j \mathbb{E}(A_{\ell,i,j}) \mathbb{E}(y_{\ell-1,j}) + 0 = 0 \quad (\because \text{independence})$$

$$\Rightarrow \mathbb{E}(y_L) = \mathbb{E}(y_{L,1}) = 0$$

$$\text{Var}(y_{\ell,i}) = \text{Var}\left(\sum_j A_{\ell,i,j} y_{\ell-1,j} + b_i\right)$$

$$= \sum_j \text{Var}(A_{\ell,i,j} y_{\ell-1,j}) + \text{Var}(b_i) \quad (\because \text{independence})$$

$$= \sum_j \text{Var}(A_{\ell,i,j}) \text{Var}(y_{\ell-1,j}) + \text{Var}(b_i) \quad (\because \text{independence}) \quad (\text{Var}(y_{\ell-1,j}) \text{ 은 } j \text{ 에 따라 같지 않음})$$

$$= n_{\ell-1} \left(\frac{1}{n_{\ell-1}} \cdot \left(\sqrt{\frac{2}{n_{\ell-1}}} \right)^2 \right) \text{Var}(y_{\ell-1,1}) + \frac{1}{n_{\ell-1}} \left(\sqrt{\frac{2}{n_{\ell-1}}} \right)^2 = \frac{1}{3} \text{Var}(y_{\ell-1,1}) + \frac{1}{3n_{\ell-1}}$$

$$\circ \text{Var}(y_L) = \text{Var}(y_{L,1})$$

$$= \frac{1}{3} \text{Var}(y_{L-1,1}) + \frac{1}{3n_{L-1}}$$

$$= \frac{1}{3} \left(\frac{1}{3} \text{Var}(y_{L-2,1}) + \frac{1}{3n_{L-2}} \right) + \frac{1}{3n_{L-1}}$$

$$= \dots$$

$$= \left(\frac{1}{3} \right)^L \text{Var}(y_{0,1}) + \sum_{i=0}^{L-1} \left(\frac{1}{3} \right)^{L-i} \frac{1}{n_i}$$

$$= \left(\frac{1}{3} \right)^L + \sum_{i=0}^{L-1} \left(\frac{1}{3} \right)^{L-i} \frac{1}{n_i} //$$

→ 이항식 3/4항에 의해
공분산 생각하면 자연스럽다.

Problem 3: Backprop with convolutions. Consider 1D convolutions with single input and output channels, stride 1, and padding 0. Let $w_1, \dots, w_L$ be convolutional filters with sizes $f_1, \dots, f_L$. Let $A_{w_\ell} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$, where $n_\ell = n_{\ell-1} - f_\ell + 1$, be the matrix representing convolution with w_ℓ , i.e., multiplication by A_{w_ℓ} is equivalent to convolution with w_ℓ , for $\ell = 1, \dots, L$. Let $\sigma: \mathbb{R} \rightarrow \mathbb{R}$ be a differentiable activation function. Consider the convolutional neural network

$$\begin{aligned} y_L &= \sigma(A_{w_L} y_{L-1} + b_L) \\ y_{L-1} &= \sigma(A_{w_{L-1}} y_{L-2} + b_{L-1}) \\ &\vdots \\ y_2 &= \sigma(A_{w_2} y_1 + b_2) \\ y_1 &= \sigma(A_{w_1} x + b_1), \end{aligned}$$

where $x \in \mathbb{R}^{n_0}$, $b_\ell \in \mathbb{R}^{n_\ell}$, and $n_L = 1$. Assume x is fixed and $y_1, \dots, y_L$ have been computed in a forward pass. For notational convenience, define $x = y_0$.

Find formulae for

$$\frac{\partial y_L}{\partial b_\ell}, \quad \frac{\partial y_L}{\partial w_\ell}$$

for $\ell = 1, \dots, L$ and describe how to compute them using backpropagation. As discussed in homework 1, forming the full matrix A_{w_ℓ} is wasteful and should be avoided. In the description, make clear when matrix-vector or vector-matrix products with respect to A_{w_i} or $A_{w_i}^T$ are used.

Back-propagation은 output stream $\frac{\partial y_L}{\partial y_\ell}$ 을 feedback으로, $y_\ell^* = A_{w_\ell} y_{\ell-1} + b_\ell$ output value,

$$\frac{\partial y_L}{\partial y_\ell} = \frac{\partial y_L}{\partial y_{\ell+1}} \frac{\partial y_{\ell+1}}{\partial y_\ell} = \left(\frac{\partial y_L}{\partial y_{\ell+1}} \text{diag}(\sigma'(y_{\ell+1}^*)) \right) A_{w_{\ell+1}} \quad \text{①}$$

$$\frac{\partial y_L}{\partial b_\ell} = \frac{\partial y_L}{\partial y_\ell} \cdot \frac{\partial y_\ell}{\partial b_\ell} = \frac{\partial y_L}{\partial y_\ell} \cdot \text{diag}(\sigma'(y_\ell^*))$$

$$\frac{\partial y_L}{\partial w_\ell} = \frac{\partial y_L}{\partial y_\ell} \frac{\partial y_\ell}{\partial w_\ell} = \frac{\partial y_L}{\partial y_\ell} \cdot \frac{\partial y_\ell}{\partial y_{\ell-1}^*} \frac{\partial y_{\ell-1}^*}{\partial w_\ell} = \frac{\partial y_L}{\partial y_\ell} \cdot \text{diag}(\sigma'(y_\ell^*)) \frac{\partial y_{\ell-1}^*}{\partial w_\ell}$$

$$\text{then } \frac{\partial y_{\ell-1}^*}{\partial w_{\ell-1,i}} = \begin{pmatrix} y_{\ell-1,i} \\ \vdots \\ y_{\ell-1,i+n_{\ell-1}-1} \end{pmatrix} \quad \text{②}$$



$$\frac{\partial y_{\ell-1}^*}{\partial w_\ell} = \begin{pmatrix} y_{\ell-1,1} & \dots & y_{\ell-1,f_\ell} \\ \vdots & \ddots & \vdots \\ y_{\ell-1,n_\ell} & \dots & y_{\ell-1,n_\ell+f_\ell-1} \end{pmatrix} \quad (n_{\ell-1} - n_\ell = f_\ell - 1) \quad \text{③}$$

또한 forward-mode에 의해 y_ℓ^*, y_ℓ 등 값 계산됨, 이를 backward-mode에 의해 $\frac{\partial y_L}{\partial y_\ell}$ 이 어떤 값이 계산되는지 구함.

①과 같은 방식의 곱셈 과정은 1D convolutional filter로 구현될 수 있음, 각각(각각)은 $y^T A_{w_{\ell+1}}$ 과 곱셈 가능함, 이는 $y^T (A_{w_{\ell+1}}^T y)$ 와 같고, 이는 y 가 $f_\ell - 1$ 의 padding을 준 다음 stride 1로 y 를 (필터의 크기만큼)의 슬라이딩 윈도우를 움직여,

$(A_{w_{\ell+1}}^T y)^T$ 와 같고, 이는 y 가 $f_\ell - 1$ 의 padding을 준 다음 stride 1로 y 를 (필터의 크기만큼)의 슬라이딩 윈도우를 움직여,

Problem 4: Change of variables formula for Gaussians. If $\varphi: \mathbb{R}^n \rightarrow \mathbb{R}^n$ is a one-to-one differentiable function, $Y = \varphi(X)$, and Y is a continuous random variable with density function p_Y , then X is a continuous random variable with density function

$$p_X(x) = p_Y(\varphi(x)) \left| \det \frac{\partial \varphi}{\partial x}(x) \right|.$$

Let $Y \in \mathbb{R}^n$ be a continuous random vector with density

$$p_Y(y) = \frac{1}{(2\pi)^{n/2}} e^{-\frac{1}{2}\|y\|^2},$$

i.e., $Y \sim \mathcal{N}(0, I)$. Let $X = AY + b$ with an invertible matrix $A \in \mathbb{R}^{n \times n}$ and a vector $b \in \mathbb{R}^n$. Define $\Sigma = AA^T$. Show that X is a continuous random vector with density

$$p_X(x) = \frac{1}{\sqrt{(2\pi)^n \det \Sigma}} e^{-\frac{1}{2}(x-b)^T \Sigma^{-1}(x-b)}.$$

$$Y = A^{-1}(X-b) = \varphi(X) \Rightarrow \frac{\partial \varphi}{\partial X} = A^{-1}$$

$$\Sigma = AA^T \Rightarrow \det \Sigma = (\det A)^2, \quad |\det A| = \sqrt{\det \Sigma}, \quad |\det A^{-1}| = 1/\sqrt{\det \Sigma}$$

$$p_X(x) = p_Y(\varphi(x)) \left| \det \frac{\partial \varphi}{\partial x}(x) \right| = p_Y(\varphi(x)) |\det A^{-1}|$$

$$\rightarrow (A^{-1})^T A^{-1} = (AA^T)^{-1} = \Sigma^{-1}$$

$$= \frac{1}{(2\pi)^{n/2}} e^{-\frac{1}{2}\|A^{-1}(x-b)\|^2} \frac{1}{\sqrt{\det \Sigma}} = \frac{1}{(2\pi)^{n/2}} e^{-\frac{1}{2}(x-b)^T (A^{-1})^T A^{-1} (x-b)} \frac{1}{\sqrt{\det \Sigma}}$$

$$= \frac{1}{\sqrt{(2\pi)^n \det \Sigma}} e^{-\frac{1}{2}(x-b)^T \Sigma^{-1}(x-b)} //$$

Problem 5: D_{KL} of continuous random variables. The KL-divergence between continuous random variables $X \sim f$ and $Y \sim g$, where f and g are probability density functions in $\mathbb{R}^d$, is

$$D_{\text{KL}}(X \| Y) = \int_{\mathbb{R}^d} f(x) \log \left(\frac{f(x)}{g(x)} \right) dx.$$

(a) Show that

$$D_{\text{KL}}(X \| Y) \geq 0.$$

($\because -\log$, convex, Jensen)

$$D_{\text{KL}}(X \| Y) = \int_{\mathbb{R}^d} f(x) \log \frac{f(x)}{g(x)} dx = \int_{\mathbb{R}^d} f(x) \left(-\log \frac{g(x)}{f(x)} \right) dx \geq \int_{\mathbb{R}^d} -\log \left(f(x) \cdot \frac{g(x)}{f(x)} \right) dx = \int_{\mathbb{R}^d} -\log g(x) dx \geq 0 //$$

(b) Show that if $X = (X_1, \dots, X_d)$ is a continuous random variable such that $X_1, \dots, X_d$ are independent and $Y = (Y_1, \dots, Y_d)$ is a continuous random variable such that $Y_1, \dots, Y_d$ are independent, then

$$D_{\text{KL}}(X \| Y) = D_{\text{KL}}(X_1 \| Y_1) + \dots + D_{\text{KL}}(X_d \| Y_d).$$

$\log \frac{f(x)}{g(x)} = \dots + \log \frac{f(x_d)}{g(x_d)}$

$$D_{\text{KL}}(X \| Y) = \int_{\mathbb{R}^d} f(x) \log \frac{f(x)}{g(x)} dx = \int_{\mathbb{R}^d} f(x_1) \dots f(x_d) \left(\log \frac{f(x_1) \dots f(x_d)}{g(x_1) \dots g(x_d)} \right) dx_1 \dots dx_d$$

($\because$ independence)

$$= \sum_{i=1}^d \int_{\mathbb{R}^d} f(x_1) \dots f(x_d) \log \frac{f(x_i)}{g(x_i)} dx_1 \dots dx_d$$

$$= \sum_{i=1}^d \left(\int_{\mathbb{R}} f(x_i) \log \frac{f(x_i)}{g(x_i)} dx_i \right) \prod_{j \neq i} \left(\int_{\mathbb{R}} f(x_j) dx_j \right) = \sum_{i=1}^d \int_{\mathbb{R}} f(x_i) \log \frac{f(x_i)}{g(x_i)} dx_i = \sum_{i=1}^d D_{\text{KL}}(X_i \| Y_i) //$$

($\because$ independence)

Problem 6: D_{KL} of Gaussian random variables. Let $\mathcal{N}(\mu, \Sigma)$ denote the Gaussian distribution with mean μ and covariance Σ . So if $X \sim \mathcal{N}(\mu, \Sigma)$, then

$$\mathbb{E}[X] = \mu, \quad \mathbb{E}[(X - \mu)(X - \mu)^T] = \Sigma. \quad \Rightarrow \quad \mathbb{E}[\text{tr}(XX^T)]$$

$$= \text{tr}(\mathbb{E}(XX^T)) = \text{tr}(\Sigma + X\mu^T) = \text{tr}(\Sigma + \mu\mu^T)$$

Show that

$$D_{KL}(\mathcal{N}(\mu_0, \Sigma_0) \| \mathcal{N}(\mu_1, \Sigma_1)) = \frac{1}{2} \left(\text{tr}(\Sigma_1^{-1}\Sigma_0) + (\mu_1 - \mu_0)^T \Sigma_1^{-1}(\mu_1 - \mu_0) - d + \log \left(\frac{\det \Sigma_1}{\det \Sigma_0} \right) \right),$$

where d is the underlying dimension of the random variables $\mathcal{N}(\mu_0, \Sigma_0)$ and $\mathcal{N}(\mu_1, \Sigma_1)$.

$$D_{KL}(\mathcal{N}(\mu_0, \Sigma_0) \| \mathcal{N}(\mu_1, \Sigma_1)) = \mathbb{E}_{P_0}(\log P_0 - \log P_1)$$

$$= \frac{1}{2} \mathbb{E}_{P_0} \left[-(X - \mu_0)^T \Sigma_0^{-1} (X - \mu_0) + (X - \mu_1)^T \Sigma_1^{-1} (X - \mu_1) - \log \det \Sigma_0 + \log \det \Sigma_1 \right]$$

$$= \frac{1}{2} \log \frac{\det \Sigma_1}{\det \Sigma_0} + \frac{1}{2} \mathbb{E}_{P_0} \left[-(X - \mu_0)^T \Sigma_0^{-1} (X - \mu_0) + (X - \mu_1)^T \Sigma_1^{-1} (X - \mu_1) \right]$$

$$= \frac{1}{2} \log \frac{\det \Sigma_1}{\det \Sigma_0} + \frac{1}{2} \mathbb{E}_{P_0} \left[-\text{tr}(\Sigma_0^{-1} (X - \mu_0)(X - \mu_0)^T) + \text{tr}(\Sigma_1^{-1} (X - \mu_1)(X - \mu_1)^T) \right]$$

$$\because \langle U, W \rangle = U^T W = \text{tr}(UW^T)$$

$$= \frac{1}{2} \log \frac{\det \Sigma_1}{\det \Sigma_0} + \frac{1}{2} \left[-\text{tr}(\Sigma_0^{-1} \mathbb{E}_{P_0}((X - \mu_0)(X - \mu_0)^T)) + \text{tr}(\Sigma_1^{-1} \mathbb{E}_{P_0}((X - \mu_1)(X - \mu_1)^T)) \right]$$

$$= \frac{1}{2} \log \frac{\det \Sigma_1}{\det \Sigma_0} + \frac{1}{2} \left[-\text{tr}(\Sigma_0^{-1} \Sigma_0) + \text{tr}(\Sigma_1^{-1} (\Sigma_0 + \mu_0 \mu_0^T - 2\mu_0 \mu_1^T + \mu_1 \mu_1^T)) \right]$$

$$= \frac{1}{2} \left(\log \frac{\det \Sigma_1}{\det \Sigma_0} - d + \text{tr}(\Sigma_1^{-1} \Sigma_0) + \text{tr}(\Sigma_1^{-1} (\mu_0 \mu_0^T - 2\mu_0 \mu_1^T + \mu_1 \mu_1^T)) \right)$$

$$= \frac{1}{2} \left(\log \frac{\det \Sigma_1}{\det \Sigma_0} - d + \text{tr}(\Sigma_1^{-1} \Sigma_0) + \text{tr}(\Sigma_1^{-1} (\mu_0 - \mu_1)(\mu_0 - \mu_1)^T) \right)$$

$$= \frac{1}{2} \left(\log \frac{\det \Sigma_1}{\det \Sigma_0} - d + \text{tr}(\Sigma_1^{-1} \Sigma_0) + (\mu_0 - \mu_1)^T \Sigma_1^{-1} (\mu_0 - \mu_1) \right)$$